

Used Car Price Prediction using Machine Learning

Predicting resale value of cars using data analysis and ML models

Yogesh · Aspiring Data Scientist



What Is This Project?



The Goal

Build an ML model that predicts the **resale price of used cars** using historical data — helping buyers and sellers make smarter decisions.

Why It Matters

The used car market is growing fast, but manual pricing is unreliable. ML automates fair price estimation for platforms like **OLX, Cars24, and CarDekho**.



PROBLEM STATEMENT

The Pricing Problem

Used car prices depend on **many overlapping factors** — year, mileage, engine power, fuel type, transmission, and more. Predicting an accurate price manually is nearly impossible at scale.



Complex Inputs

10+ features influence price simultaneously



Inaccurate Estimates

Manual pricing leads to buyer/seller distrust



ML Solution

Regression models learn patterns from data

Dataset at a Glance

Dataset Details

7,253 rows × 13 columns of real-world used car listings. A mix of numerical and categorical features — ideal for regression.

- Year, Kilometers Driven, Seats
- Fuel Type, Transmission, Mileage
- Engine (CC), Power (bhp), Price



Why this dataset? Large, real-world, multi-feature — perfect for training and uncovering pricing patterns.

Feature	Type	Example
Year	Numeric	2016
Fuel_Type	Categorical	Diesel
Transmission	Categorical	Manual
Mileage	Numeric	18.5 km/l
Engine	Numeric	1248 CC
Power	Numeric	74 bhp
Price	Target	₹6.5 Lakhs

Libraries Used for EDA

Importing Useful Libraries for EDA

```
# Importing necessary libraries  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
import warnings  
warnings.filterwarnings('ignore')
```

Reading CSV

```
# Loading the dataset  
ucd = pd.read_csv('used_cars_data.csv')
```

Basic Data Understanding

```
# Basic Data Understanding
print("Dataset Shape:", ucd.shape)
print("\nFirst Few Rows:")
display(ucd.head())
print("\nData Types:")
display(ucd.dtypes)
print("\nDataset Info:")
ucd.info()
```

Dataset Shape: (7253, 13)

- Get Information about Data

```
Dataset Info:
<class 'pandas.DataFrame'>
RangeIndex: 7253 entries, 0 to 7252
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   S.No.                 7253 non-null  int64
1   Name                  7253 non-null  str
2   Location              7253 non-null  str
3   Year                  7253 non-null  int64
4   Kilometers_Driven     7253 non-null  int64
5   Fuel_Type             7253 non-null  str
6   Transmission          7253 non-null  str
7   Owner_Type            7253 non-null  str
8   Mileage                7251 non-null  str
9   Engine                7207 non-null  str
10  Power                 7207 non-null  str
11  Seats                 7200 non-null  float64
12  Price                 7253 non-null  float64
dtypes: float64(2), int64(3), str(8)
memory usage: 736.8 KB
```

- Get the first 10 Rows of Data

First Few Rows:													
S.No.		Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	Price
0	0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	26.6 km/kg	998 CC	58.16 bhp	5.0	1.75
1	1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	41000	Diesel	Manual	First	19.67 kmpl	1582 CC	126.2 bhp	5.0	12.50
2	2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	18.2 kmpl	1199 CC	88.7 bhp	5.0	4.50
3	3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	20.77 kmpl	1248 CC	88.76 bhp	7.0	6.00
4	4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	15.2 kmpl	1968 CC	140.8 bhp	5.0	17.74
5	5	Hyundai EON LPG Era Plus Option	Hyderabad	2012	75000	LPG	Manual	First	21.1 km/kg	814 CC	55.2 bhp	5.0	2.35
6	6	Nissan Micra Diesel XV	Jaipur	2013	86999	Diesel	Manual	First	23.08 kmpl	1461 CC	63.1 bhp	5.0	3.50
7	7	Toyota Innova Crysta 2.8 GX AT 8S	Mumbai	2016	36000	Diesel	Automatic	First	11.36 kmpl	2755 CC	171.5 bhp	8.0	17.50
8	8	Volkswagen Vento Diesel Comfortline	Pune	2013	64430	Diesel	Manual	First	20.54 kmpl	1598 CC	103.6 bhp	5.0	5.20
9	9	Tata Indica Vista Quadrajet LS	Chennai	2012	65932	Diesel	Manual	Second	22.3 kmpl	1248 CC	74 bhp	5.0	1.95

- Statistical Summary of Numerical Features

```
# DESCRIPTIVE STATISTICS OF THE DATASET  
ucd.describe()
```

	S.No.	Year	Kilometers_Driven	Seats	Price
count	7253.000000	7253.000000	7.253000e+03	7200.000000	7253.000000
mean	3626.000000	2013.365366	5.869906e+04	5.279722	8.826235
std	2093.905084	3.254421	8.442772e+04	0.811660	10.293313
min	0.000000	1996.000000	1.710000e+02	0.000000	0.440000
25%	1813.000000	2011.000000	3.400000e+04	5.000000	3.850000
50%	3626.000000	2014.000000	5.341600e+04	5.000000	5.640000
75%	5439.000000	2016.000000	7.300000e+04	5.000000	8.400000
max	7252.000000	2019.000000	6.500000e+06	10.000000	160.000000

Cleaning & Preparing the Data



A clean 4-step pipeline transforms raw data into model-ready input.

Missing Values & Outliers

Rows with missing critical values were removed. Median imputation handled mileage gaps. **IQR method** ($Q1$, $Q3$, $IQR = Q3 - Q1$) detected and removed extreme outliers from numerical columns.



Quick Definitions:

Outlier: Extreme value outside normal range

IQR: Interquartile Range = $Q3 - Q1$

Encoding: Converts categories to numbers

Scaling: Normalizes feature ranges

- Fixing Data types and Name of the Features

```
# FIXING DATA TYPES AND CORRECTING NAMES OF THE FEATURES
# Drop S.No. as it is not useful for analysis
ucd.drop(columns=['S.No.'], inplace=True)

# Remove 'kmpl' from Mileage column and convert to numeric
ucd['Mileage'] = ucd['Mileage'].str.extract(r'(\d+\.\d*)').astype(float)
ucd.rename(columns={'Mileage': 'Mileage_Kmpl'}, inplace=True)

# Remove 'CC' from Engine column and convert to numeric
ucd['Engine'] = ucd['Engine'].str.extract(r'(\d+\.\d*)').astype(float)
ucd.rename(columns={'Engine': 'Engine_CC'}, inplace=True)

# Remove 'bhp' from Power column and convert to numeric
ucd['Power'] = ucd['Power'].str.extract(r'(\d+\.\d*)').astype(float)
ucd.rename(columns={'Power': 'Power_bhp'}, inplace=True)

# Convert Price from Lakh to actual rupees (multiply by 100,000)
ucd['Price'] = ucd['Price'] * 100000

# Extract Brand from Name
ucd['Brand'] = ucd['Name'].str.split().str[0]

# Reorder columns to place Brand after Name
columns = list(ucd.columns)
columns.remove('Brand')
columns.insert(1, 'Brand')
ucd = ucd[columns]
```

- Taking a look on data after Fixing

```
ucd.head(10)
```

	Name	Brand	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage_Kmpl	Engine_CC	Power_bhp	Seats	Price
0	Maruti Wagon R LXI CNG	Maruti	Mumbai	2010	72000	CNG	Manual	First	26.60	998.0	58.16	5.0	175000.0
1	Hyundai Creta 1.6 CRDi SX Option	Hyundai	Pune	2015	41000	Diesel	Manual	First	19.67	1582.0	126.20	5.0	1250000.0
2	Honda Jazz V	Honda	Chennai	2011	46000	Petrol	Manual	First	18.20	1199.0	88.70	5.0	450000.0
3	Maruti Ertiga VDI	Maruti	Chennai	2012	87000	Diesel	Manual	First	20.77	1248.0	88.76	7.0	600000.0
4	Audi A4 New 2.0 TDI Multitronic	Audi	Coimbatore	2013	40670	Diesel	Automatic	Second	15.20	1968.0	140.80	5.0	1774000.0
5	Hyundai EON LPG Era Plus Option	Hyundai	Hyderabad	2012	75000	LPG	Manual	First	21.10	814.0	55.20	5.0	235000.0
6	Nissan Micra Diesel XV	Nissan	Jaipur	2013	86999	Diesel	Manual	First	23.08	1461.0	63.10	5.0	350000.0
7	Toyota Innova Crysta 2.8 GX AT 8S	Toyota	Mumbai	2016	36000	Diesel	Automatic	First	11.36	2755.0	171.50	8.0	1750000.0
8	Volkswagen Vento Diesel Comfortline	Volkswagen	Pune	2013	64430	Diesel	Manual	First	20.54	1598.0	103.60	5.0	520000.0
9	Tata Indica Vista Quadrajet LS	Tata	Chennai	2012	65932	Diesel	Manual	Second	22.30	1248.0	74.00	5.0	195000.0

Missing Values Analysis

- Missing values are empty or unavailable data points in a dataset. They occur when information is not recorded, lost, or unknown.

```
print("Missing Values:")
missing_df = pd.DataFrame({
    'Column': ucd.columns,
    'Missing_Count': ucd.isnull().sum(),
    'Missing_Percentage': (ucd.isnull().sum() / len(ucd) * 100).round(2)
})
print(missing_df[missing_df['Missing_Count'] > 0].sort_values('Missing_Count', ascending=False))
```

Missing Values:

	Column	Missing_Count	Missing_Percentage
Power_bhp	Power_bhp	175	2.41
Seats	Seats	53	0.73
Engine_CC	Engine_CC	46	0.63
Mileage_Kmpl	Mileage_Kmpl	2	0.03

- Handling Missing values

```
# HANDLING MISSING VALUES
# Drop rows with missing values in Seats, Power_bhp, Engine_CC, and Mileage_Kmpl
ucd.dropna(subset=['Seats', 'Power_bhp', 'Engine_CC', 'Mileage_Kmpl'], inplace=True)

# Final dataset shape after handling missing values
print(f"Dataset shape after handling missing values: {ucd.shape}")

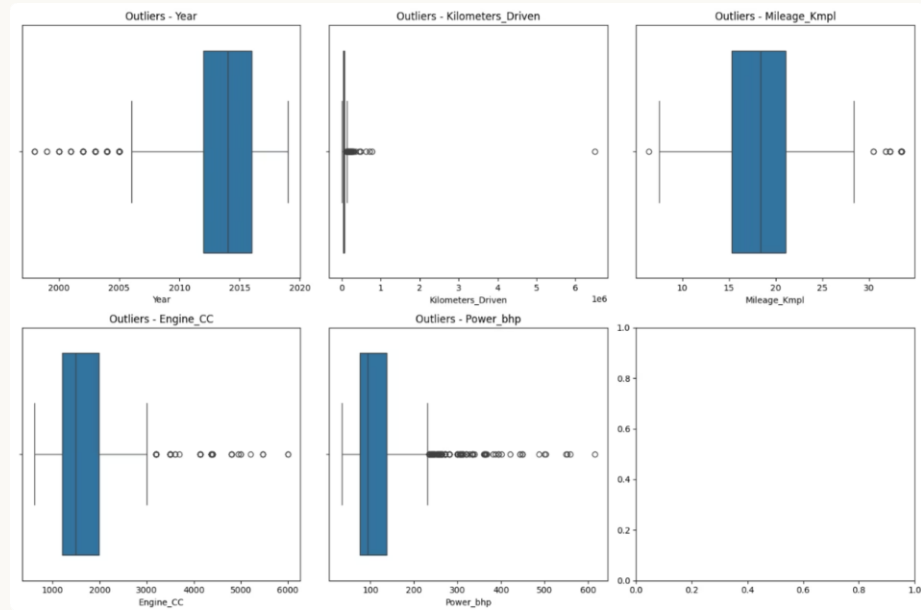
# Fill Median values Where Mileage is 0
mileage_median = ucd[ucd['Mileage_Kmpl'] > 0]['Mileage_Kmpl'].median()
ucd.loc[ucd['Mileage_Kmpl'] == 0, 'Mileage_Kmpl'] = mileage_median
print(f"Filled {(ucd['Mileage_Kmpl'] == 0).sum()} zero values in Mileage_Kmpl with median: {mileage_median:.2f}")
```

```
Dataset shape after handling missing values: (7073, 13)
Filled 0 zero values in Mileage_Kmpl with median: 18.40
```

Outlier Analysis

Outliers are extreme values that differ significantly from normal data. They were handled using the IQR method to improve model performance and prediction accuracy.

```
# OUTLIER HANDLING
columns_to_check = ['Year', 'Kilometers_Driven', 'Mileage_Kmpl', 'Engine_CC', 'Power_bhp']
# Boxplot to spot outliers visually in a 2x3 subplot
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()
for i, col in enumerate(columns_to_check):
    sns.boxplot(x=ucd[col], ax=axes[i])
    axes[i].set_title(f'Outliers - {col}')
plt.tight_layout()
plt.show()
```



Handling Outliers - Using IQR Method

- IQR is a method used to identify outliers by measuring the spread of the middle 50% of data. Values outside the calculated range are considered outliers.

```
# IQR Method
for col in columns_to_check:
    Q1 = ucd[col].quantile(0.25)
    Q3 = ucd[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    ucd = ucd[(ucd[col] >= lower) | (ucd[col] <= upper)]
```

- Q1 (First Quartile)**
Q1 is the value below which 25% of the data lies. It represents the lower part of the dataset.
- Q3 (Third Quartile)**
Q3 is the value below which 75% of the data lies. It represents the upper part of the dataset.
- Lower Limit**
The lower boundary used to detect outliers. Any value smaller than this limit is considered an outlier. $Lower\ Limit = Q1 - 1.5 \times IQR$
 $Lower\ Limit = Q1 - 1.5 \times IQR$
- Upper Limit**
The upper boundary used to detect outliers. Any value greater than this limit is considered an outlier. $Upper\ Limit = Q3 + 1.5 \times IQR$
 $Upper\ Limit = Q3 + 1.5 \times IQR$

Univariate Analysis

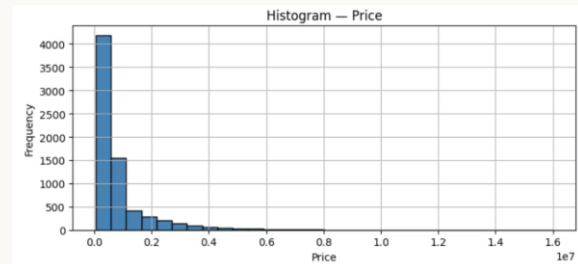
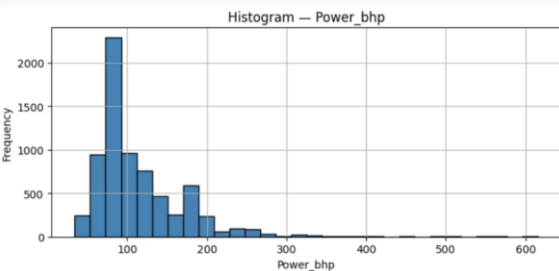
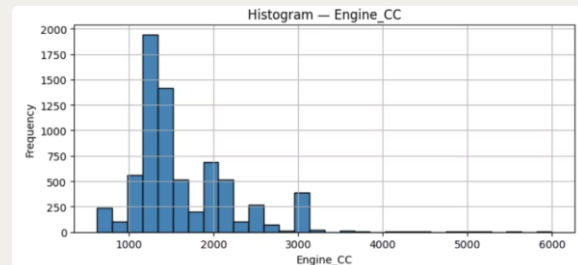
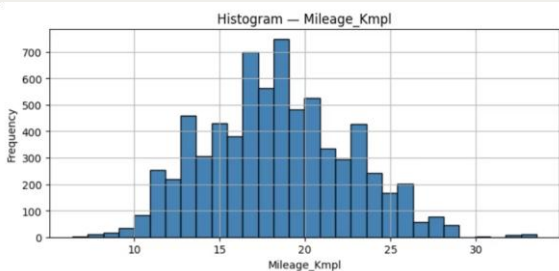
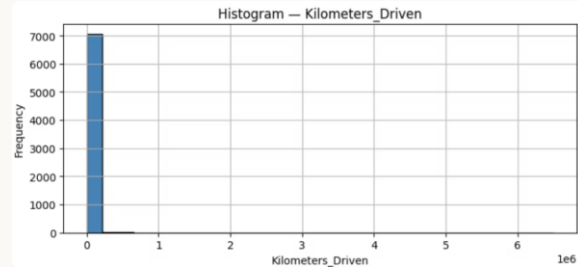
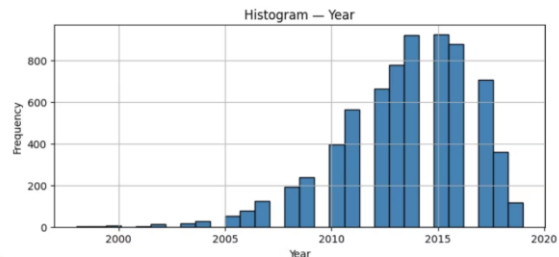
Process of analyzing a single variable to understand its distribution, patterns, and behavior.

```
numeric_cols = ['Year', 'Kilometers_Driven', 'Mileage_Kmpl', 'Engine_CC', 'Power_bhp', 'Price']
for col in numeric_cols:
    fig, axes = plt.subplots(1, 2, figsize=(15, 4))

    # Histogram
    ucd[col].hist(ax=axes[0], bins=30, color='steelblue', edgecolor='black')
    axes[0].set_title(f'Histogram - {col}')
    axes[0].set_xlabel(col)
    axes[0].set_ylabel('Frequency')

    plt.suptitle(col, fontsize=13, fontweight='bold')
    plt.tight_layout()
    plt.show()
```


Distribution of Numerical Features



- Distribution of Categorical Features

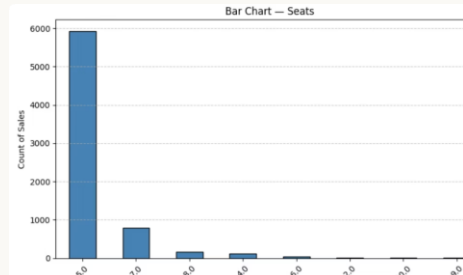
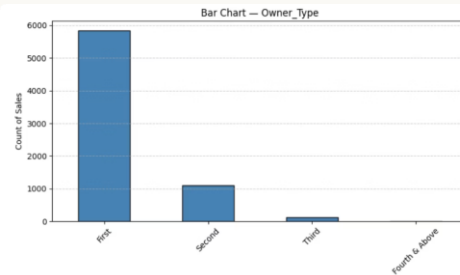
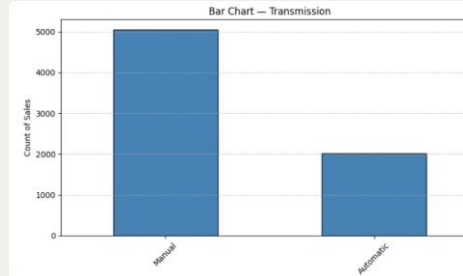
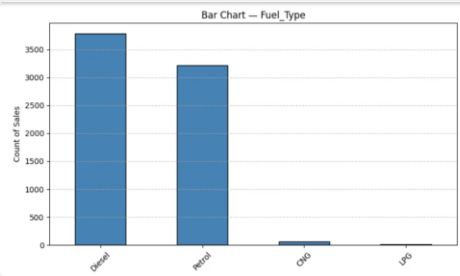
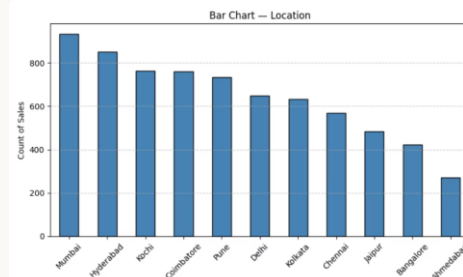
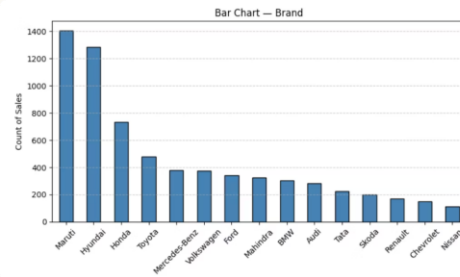
```
categorical_cols = ucd[['Brand', 'Location', 'Fuel_Type', 'Transmission', 'Owner_Type', 'Seats']]
for col in categorical_cols:
    n_unique = ucd[col].nunique()
    print(f"\n{col} - {n_unique} unique values")
    print(ucd[col].value_counts().head(10))

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# Bar chart (top 15)
ucd[col].value_counts().head(15).plot(
    kind='bar', ax=axes[0], color='steelblue', edgecolor='black'
)
axes[0].set_title(f'Bar Chart - {col}')
axes[0].set_xlabel('')
axes[0].tick_params(axis='x', rotation=45)
axes[0].set_ylabel('Count of Sales')
axes[0].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()
```

Distribution of Catagorical Features



Bivariate Analysis

The process of analyzing the relationship between two variables to identify patterns, correlations, or trends.

```
price_col = ucd["Price"]

price_col = 'Price'
other_num_cols = [col for col in numeric_cols if col != price_col]

for col in other_num_cols:
    fig, ax = plt.subplots(figsize=(7, 5))

    ax.scatter(
        ucd[col], ucd[price_col],
        alpha=0.5, color='steelblue',
        edgecolors='white', linewidths=0.3
    )

    # Trendline
    clean = ucd[[col, price_col]].dropna()
    m, b = np.polyfit(clean[col], clean[price_col], 1)
    ax.plot(clean[col], m * clean[col] + b, color='red', linewidth=1.5, label='Trend')
```

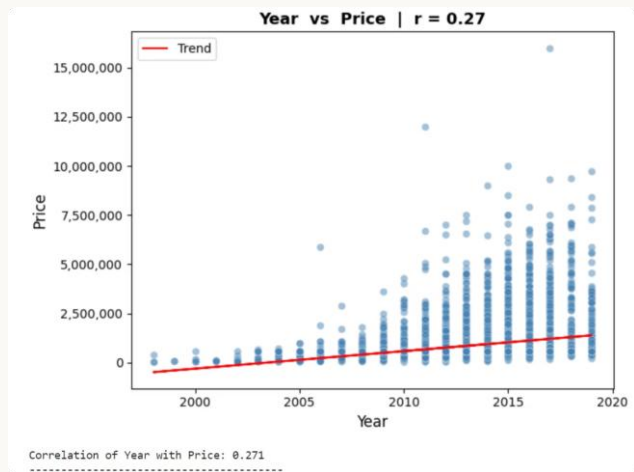
```
# Y-axis - price labels
ax.yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _ : f'{x:,0f}')
)

# Labels
ax.set_xlabel(col, fontsize=12)
ax.set_ylabel('Price', fontsize=12)

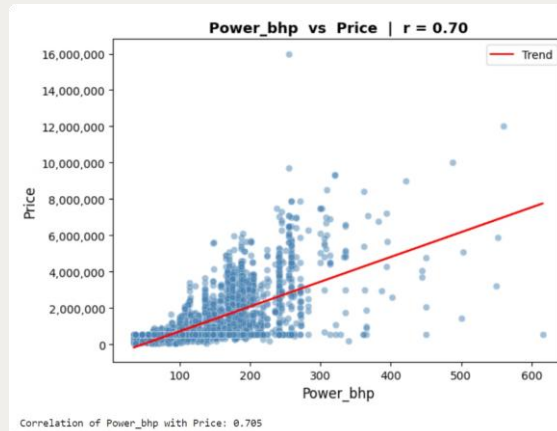
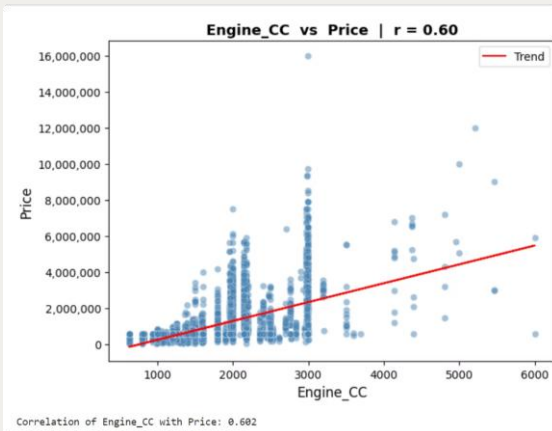
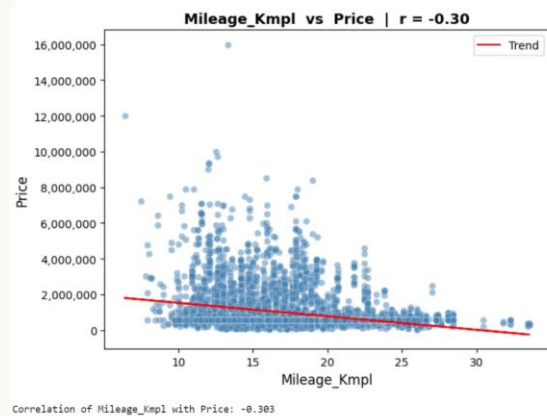
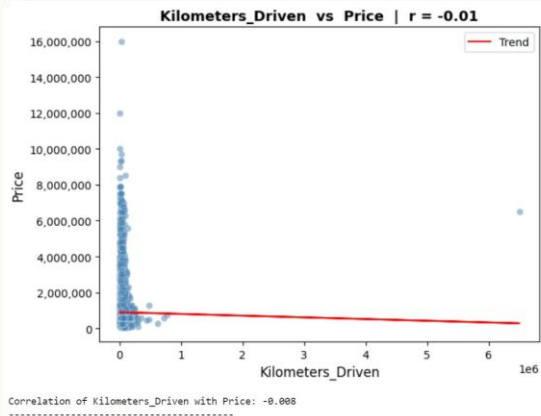
corr = clean[col].corr(clean[price_col])
ax.set_title(f'{col} vs Price | r = {corr:.2f}', fontsize=13, fontweight='bold')
ax.legend()

plt.tight_layout()
plt.show()

print(f"Correlation of {col} with Price: {corr:.3f}")
print("-" * 40)
```



Numerical Columns vs Price



Catagoricl Columns Vs Price

```
for col in categorical_cols:
    clean = ucd[[col, price_col]].dropna()

    # Keep top 10 categories by median price
    top_cats = (
        clean.groupby(col)[price_col]
        .median()
        .sort_values(ascending=False)
        .head(10)
        .index
    )
    clean = clean[clean[col].isin(top_cats)]

    # Order by median price
    order = (
        clean.groupby(col)[price_col]
        .median()
        .sort_values(ascending=False)
        .index
    )

    plt.figure(figsize=(10, 6))

    sns.violinplot(
        data=clean,
        x=col,
        y=price_col,
        order=order,
        palette='muted'
    )
```

```
# Y-axis price labels
plt.gca().yaxis.set_major_formatter(
    plt.FuncFormatter(lambda x, _ : f'{x:,.0f}')
)

# Median price annotation on each box
medians = clean.groupby(col)[price_col].median()
for j, category in enumerate(order):
    plt.text(
        j, medians[category],
        f'{medians[category]:,.0f}',
        ha='center', va='bottom',
        fontsize=9, color='black', fontweight='bold'
    )

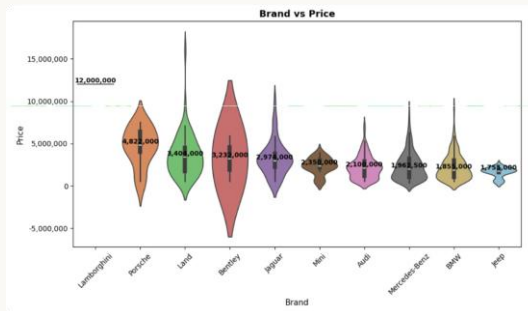
plt.title(f'{col} vs Price', fontsize=13, fontweight='bold')
plt.xlabel(col, fontsize=11)
plt.ylabel('Price', fontsize=11)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print(f"\nMedian Price by {col}:")
print(medians.reindex(order).apply(lambda x: f'{x:,.0f}'))
print("-" * 40)
```

Median Price by Brand:

Brand	
Lamborghini	12,000,000
Porsche	4,822,000
Land	3,404,000
Bentley	3,232,000
Jaguar	2,974,000
Jeep	2,974,000
Mini	2,350,000
Audi	2,100,000
Mercedes-Benz	1,962,500
BMW	1,855,000
Jeep	1,755,000

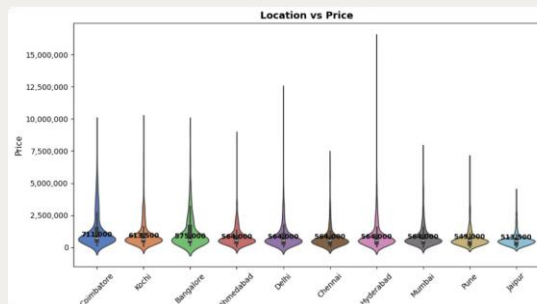
Name: Price, dtype: str



Median Price by Location:

Location	
Coimbatore	711,000
Kochi	613,500
Bangalore	575,000
Ahmedabad	564,000
Delhi	564,000
Chennai	564,000
Hyderabad	564,000
Mumbai	564,000
Pune	549,000
Jaipur	513,500

Name: Price, dtype: str



Catagoricl Columns Vs Price

Median Price by Fuel_Type:

Fuel_Type

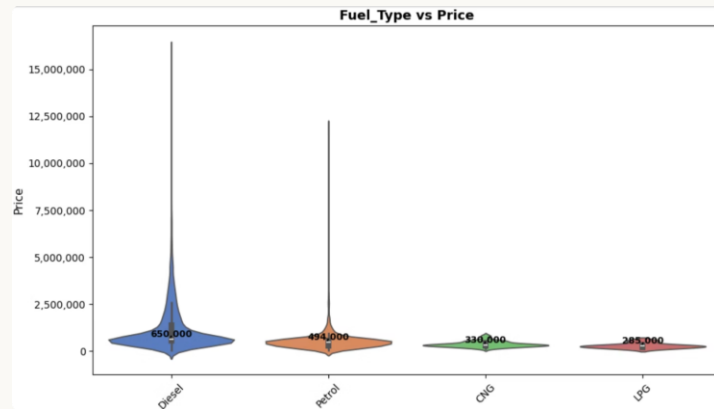
Diesel 650,000

Petrol 494,000

CNG 330,000

LPG 285,000

Name: Price, dtype: str



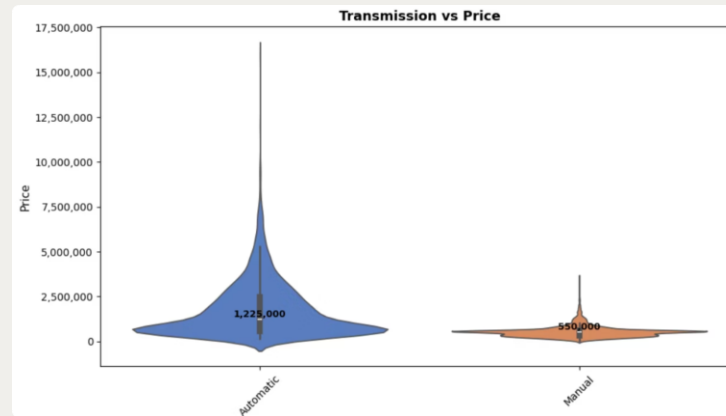
Median Price by Transmission:

Transmission

Automatic 1,225,000

Manual 550,000

Name: Price, dtype: str

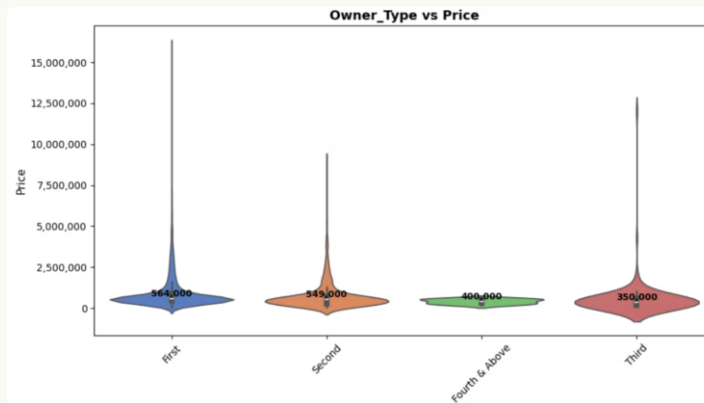


Catagorical Columns Vs Price

Median Price by Owner_Type:

Owner_Type	Median Price
First	564,000
Second	549,000
Fourth & Above	400,000
Third	350,000

Name: Price, dtype: str



Median Price by Seats:

Seats

2.0 5,500,000

7.0 900,000

6.0 583,500

4.0 564,000

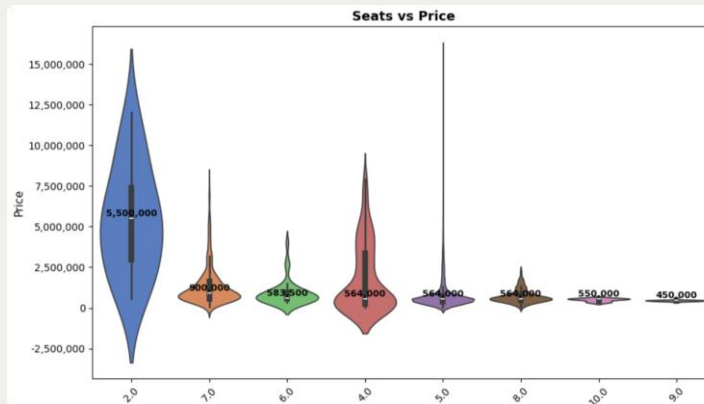
5.0 564,000

8.0 564,000

10.0 550,000

9.0 450,000

Name: Price, dtype: str



Multivariate Analysis

The process of analyzing relationships among multiple variables at the same time to understand complex patterns and interactions.

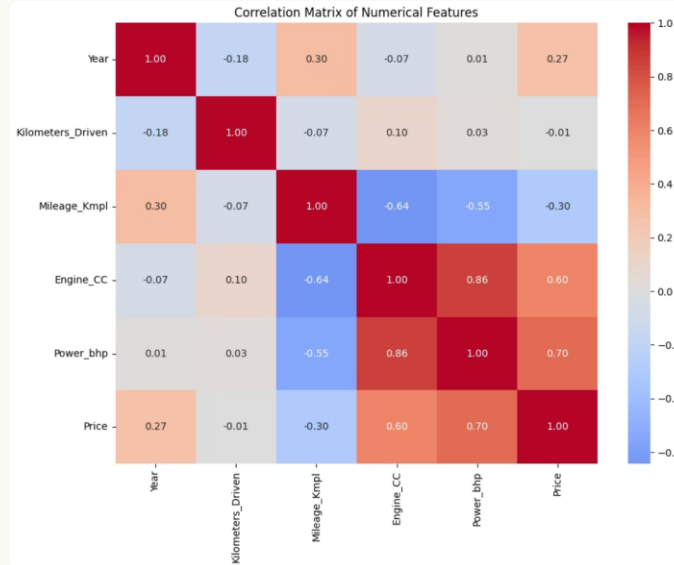
```
# Correlation Analysis
if len(ucd[numeric_cols]) > 1:
    plt.figure(figsize=(10, 8))
    sns.heatmap(ucd[numeric_cols].corr(), annot=True, cmap='coolwarm', center=0, fmt='.2f')
    plt.title('Correlation Matrix of Numerical Features')
    plt.tight_layout()
    plt.show()

print("\nHighest Correlations with Numeric Columns:")
print(ucd[numeric_cols].corr().abs().unstack().sort_values(ascending=False).head(10))
```

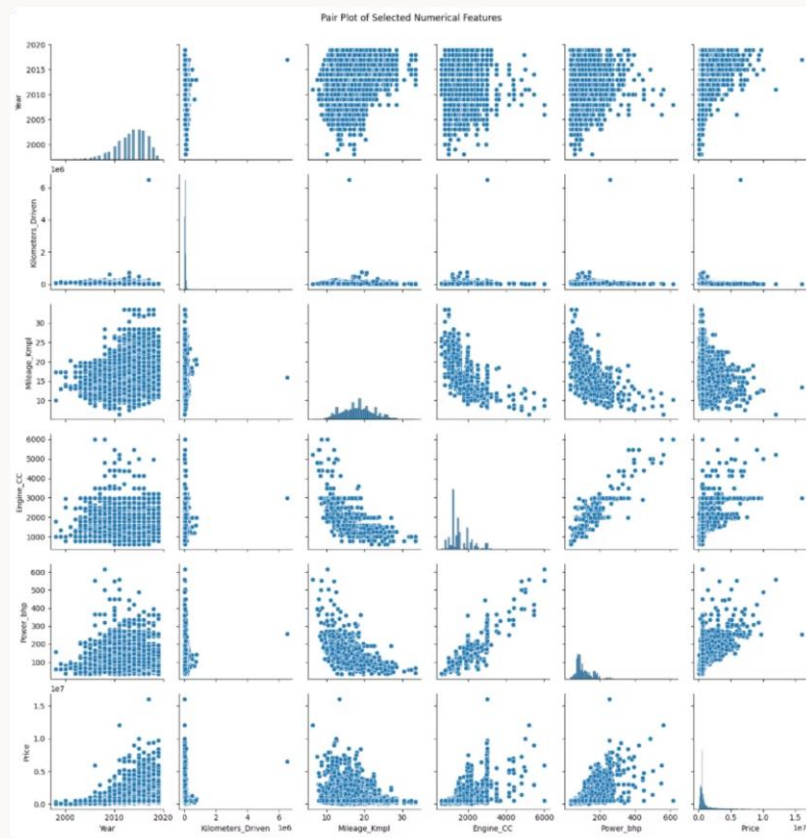
Highest Correlations with Numeric Columns:

Year	Year	1.000000
Power_bhp	Power_bhp	1.000000
Mileage_Kmpl	Mileage_Kmpl	1.000000
Kilometers_Driven	Kilometers_Driven	1.000000
Price	Price	1.000000
Engine_CC	Engine_CC	1.000000
Power_bhp	Engine_CC	0.859864
Engine_CC	Power_bhp	0.859864
Power_bhp	Price	0.704706
Price	Power_bhp	0.704706

dtype: float64



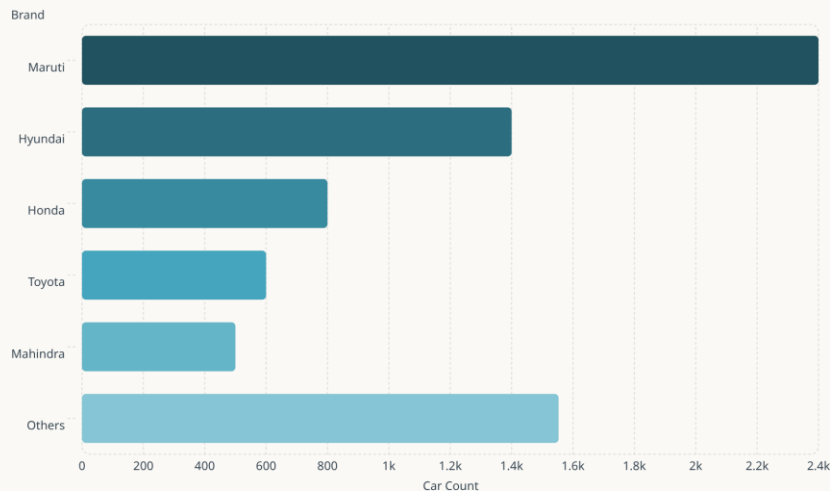
```
# Pair Plot of Numerical Features
sns.pairplot(ucd[numeric_cols])
plt.suptitle('Pair Plot of Selected Numerical Features', y=1.02)
plt.show()
```



What the Data Reveals

Key Findings

- **Maruti & Hyundai** dominate the dataset
- Diesel and Petrol are the most common fuel types
- Manual transmission cars outnumber automatic
- Price is **right-skewed** – most cars are affordable
- Engine CC and Power show **strong positive** correlation with price
- Mileage shows **negative** correlation with price



FEATURE ENGINEERING

```
# 'Name' column as it's high cardinality and not suitable for one-hot encoding in this context
ucd = ucd.drop('Name', axis=1)
ucd = ucd.drop('Seats', axis=1)

# Car Age - more useful than Year directly
ucd['Car_Age'] = 2024 - ucd['Year']

# Drop Year - no longer needed
ucd.drop(columns=['Year'], inplace=True)
```

```
ucd.head()
```

	Brand	Location	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage_Kmpl	Engine_CC	Power_bhp	Price	Car_Age
0	Maruti	Mumbai	72000	CNG	Manual	First	26.60	998.0	58.16	175000.0	14
1	Hyundai	Pune	41000	Diesel	Manual	First	19.67	1582.0	126.20	1250000.0	9
2	Honda	Chennai	46000	Petrol	Manual	First	18.20	1199.0	88.70	450000.0	13
3	Maruti	Chennai	87000	Diesel	Manual	First	20.77	1248.0	88.76	600000.0	12
4	Audi	Coimbatore	40670	Diesel	Automatic	Second	15.20	1968.0	140.80	1774000.0	11

Encoding

The process of converting categorical or text data into numerical format so Machine Learning models can understand and process it.

```
# One Hot Encoding
ucd_encoded = pd.get_dummies(ucd, columns=['Brand', 'Location', 'Fuel_Type', 'Transmission', 'Owner_Type'], drop_first=True)

ucd_encoded = ucd_encoded.astype(int)

print("Shape after encoding:", ucd.shape)
print(ucd_encoded.head())
```

```
Shape after encoding: (7073, 11)
Kilometers_Driven  Mileage_Kmpl  Engine_CC  Power_bhp  Price  Car_Age  \
0          72000          26         998         58  175000        14
1          41000          19        1582         126  1250000         9
2          46000          18        1199          88   450000        13
3          87000          20        1248          88   600000        12
4          40670          15        1968         140  1773999        11

Brand_Audi  Brand_BMW  Brand_Bentley  Brand_Chevrolet  ...  \
0           0           0             0                 0  ...
1           0           0             0                 0  ...
2           0           0             0                 0  ...
3           0           0             0                 0  ...
4           1           0             0                 0  ...

Location_Kolkata  Location_Mumbai  Location_Pune  Fuel_Type_Diesel  \
0           0                     1                 0                 0
1           0                     0                 1                 1
2           0                     0                 0                 0
3           0                     0                 0                 1
4           0                     0                 0                 1

Fuel_Type_LPG  Fuel_Type_Petrol  Transmission_Manual  \
0           0                   0                     1
1           0                   0                     1
2           0                   1                     1
3           0                   0                     1
4           0                   0                     0

Owner_Type_Fourth & Above  Owner_Type_Second  Owner_Type_Third
0                         0                   0                 0
1                         0                   0                 0
2                         0                   0                 0
3                         0                   0                 0
4                         0                   1                 0

[5 rows x 53 columns]
```

One Hot Encoding was used because categorical features had no natural order. It converts categories into separate binary columns without creating false numerical relationships.

```
# Importing necessary libraries for ML
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import StackingRegressor
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

Log transformation was applied to the target variable to reduce skewness and handle large price variations. This improved model stability and prediction accuracy.

```
# Separate features and target
X = ucd_encoded.drop(columns=['Price'])
y = ucd_encoded['Price']

y_log = np.log1p(y)
# Train on y_log instead of y
# After prediction: y_pred_actual = np.expml(y_pred)

X_train, X_test, y_train, y_test = train_test_split(X, y_log, test_size=0.3, random_state=42)

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

Shape of X_train: (4951, 52)
Shape of X_test: (2122, 52)
Shape of y_train: (4951,)
Shape of y_test: (2122,)
```

Scaling The values of X_train and X_test

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Scaling is the process of bringing numerical features into a similar range to improve model accuracy and performance.

Training & Evaluating Linear Model

```
# Initialize the Linear Regression model
lr = LinearRegression()

# Train the model
lr.fit(X_train_scaled, y_train)

# Predictions
y_pred_lr = lr.predict(X_test_scaled)

# Evaluate the model
MSE_lr = mean_squared_error(y_test, y_pred_lr)
R2_lr = r2_score(y_test, y_pred_lr)
RMSE_lr = np.sqrt(MSE_lr)

print(f"\nLinear Regression Model Performance:")
print(f"Mean Squared Error (MSE): {MSE_lr:.2f}")
print(f"RMSE: {RMSE_lr:.2f}")
print(f"R-squared (R2): {R2_lr:.2f}")

# Convert predictions and actual values back to original scale
y_test_actual = np.expml(y_test)
y_pred_actual = np.expml(y_pred_lr)

mae_rupees = mean_absolute_error(y_test_actual, y_pred_actual)
print(f"On average, the Linear model is off by: ₹{mae_rupees:.2f}")
```

Linear Regression Model Performance:

Mean Squared Error (MSE): 0.15

RMSE: 0.39

R-squared (R2): 0.76

On average, the Linear model is off by: ₹249,489.56

Training & Evaluating Random Forest Model

```
# Initialize and train Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)

# Predictions
y_pred_rf = rf.predict(X_test_scaled)

# Evaluation
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print(f"Random Forest Performance:")
print(f"RMSE: {rmse_rf:.2f}")
print(f"R2 Score: {r2_rf:.2f}")

# Convert predictions and actual values back to original scale
y_test_actual = np.expml(y_test)
y_pred_actual = np.expml(y_pred_rf)

mae_rupees = mean_absolute_error(y_test_actual, y_pred_actual)
print(f"On average, the Random Forest model is off by: ₹{mae_rupees:,.2f}")
```

Random Forest Performance:

RMSE: 0.39

R2 Score: 0.75

On average, the Random Forest model is off by: ₹240,694.51

Training & Evaluating KNN Model

```
# Initialize KNN Regressor
# n_neighbors=5 is the default
knn = KNeighborsRegressor(n_neighbors=5)

# Train the model
knn.fit(X_train_scaled, y_train)

# Predictions (in Log scale)
y_pred_knn = knn.predict(X_test_scaled)

# Evaluation
mse_knn = mean_squared_error(y_test, y_pred_knn)
rmse_knn = np.sqrt(mse_knn)
r2_knn = r2_score(y_test, y_pred_knn)

print(f"KNN Performance:")
print(f"RMSE (Log Scale): {rmse_knn:.2f}")
print(f"R2 Score: {r2_knn:.2f}")

# Convert back to actual Rupees
y_test_actual = np.expml(y_test)
y_pred_knn_actual = np.expml(y_pred_knn)
mae_knn_rupees = mean_absolute_error(y_test_actual, y_pred_knn_actual)

print(f"On average, the KNN model is off by: ₹{mae_knn_rupees:,.2f}")
```

KNN Performance:

RMSE (Log Scale): 0.47

R2 Score: 0.64

On average, the KNN model is off by: ₹307,048.19

Training & Evaluating SVR Model

```
# Initialize and train SVR
svr = SVR(kernel='rbf', C=1.0, epsilon=0.1)
svr.fit(X_train_scaled, y_train)

# Predictions (Log Scale)
y_pred_svr = svr.predict(X_test_scaled)

# Individual SVR Evaluation Metrics
mse_svr = mean_squared_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mse_svr)
r2_svr = r2_score(y_test, y_pred_svr)

# Convert back to actual Rupees for the MAE
y_test_actual = np.expml(y_test)
y_pred_svr_actual = np.expml(y_pred_svr)
mae_svr_rupees = mean_absolute_error(y_test_actual, y_pred_svr_actual)

print(f"SVR Performance:")
print(f"RMSE (Log Scale): {rmse_svr:.4f}")
print(f"R2 Score: {r2_svr:.4f}")
print(f"MAE (Rupees): ₹{mae_svr_rupees:,.2f}")
```

SVR Performance:
RMSE (Log Scale): 0.3912
R2 Score: 0.7502
MAE (Rupees): ₹214,440.55

Training & Evaluating Stacking Model

```
# 1. Define the base models
base_models = [
    ('svr', SVR(kernel='rbf', C=1.0, epsilon=0.1)),
    ('rf', RandomForestRegressor(n_estimators=100, random_state=42)),
    ('gb', GradientBoostingRegressor(n_estimators=100, max_depth=5, random_state=42))
]

# 2. Initialize Stacking Regressor
# We use Ridge (a linear model) as the final_estimator to combine predictions
stacking_reg = StackingRegressor(
    estimators=base_models,
    final_estimator=Ridge()
)

# 3. Train the model
stacking_reg.fit(X_train_scaled, y_train)

# 4. Predictions (Log Scale)
y_pred_stack = stacking_reg.predict(X_test_scaled)

# 5. Evaluation
rmse_stack = np.sqrt(mean_squared_error(y_test, y_pred_stack))
r2_stack = r2_score(y_test, y_pred_stack)

print(f"Stacking Regressor Performance:")
print(f"RMSE (Log Scale): {rmse_stack:.4f}")
print(f"R2 Score: {r2_stack:.4f}")

# Convert back to actual Rupees
y_test_actual = np.expml(y_test)
y_pred_stack_actual = np.expml(y_pred_stack)
mae_stack_rupees = mean_absolute_error(y_test_actual, y_pred_stack_actual)

print(f"On average, the Stacking model is off by: ₹{mae_stack_rupees:.2f}")
```

Stacking Regressor Performance:

RMSE (Log Scale): 0.3725

R2 Score: 0.7735

On average, the Stacking model is off by: ₹224,656.22

Five Models, One Goal



Linear Regression

Fits a straight line to data.

Baseline model. $R^2 = 0.76$, MAE $\approx$ ₹249K



Random Forest

Ensemble of trees for

robustness. $R^2 = 0.75$, MAE $\approx$ ₹240K



KNN Regressor

Predicts using nearest

neighbors. $R^2 = 0.64$, MAE $\approx$ ₹307K



SVR

Margin-based regression. **Best**

MAE $\approx$ ₹214K



Stacking Regressor

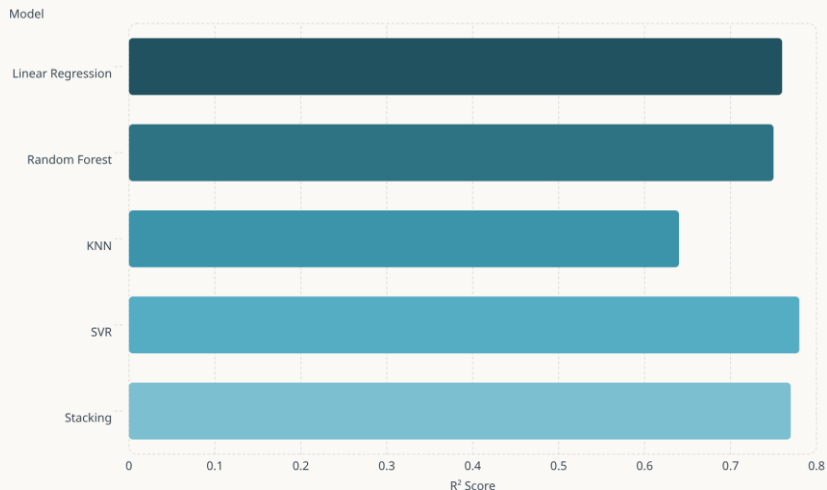
Combines base model

predictions. $R^2 = 0.77$, MAE $\approx$

₹224K

MODEL COMPARISON

Which Model Performed Best?



Key Metrics

- **SVR** — Lowest MAE (₹214K), highest R^2
- **Stacking** — Best combined R^2 (0.77)
- **KNN** — Weakest performer ($R^2 = 0.64$)



Quick Definitions:

R^2 : How well the model fits data (0–1)

MAE: Average prediction error in ₹

RMSE: Penalizes larger errors more

What We Learned

Key Insights

- **Power & Engine** most strongly impact price
- Automatic cars generally cost more
- Diesel cars have higher median prices
- Car age significantly reduces resale value

Challenges Faced

- Missing values in Mileage and Power
- Outliers in Kilometers Driven and Price
- Feature encoding for categorical variables
- Model tuning and skewed data distribution

A silver sports car, possibly a Porsche Carrera GT, is shown from a front-three-quarter view, driving on a paved road. The background is a sunset or sunrise scene with a warm orange and yellow sky and a flat horizon. The car's license plate reads "MOT C SITE".

CONCLUSION

Thank You

This project demonstrates a **complete ML pipeline** — from raw data to accurate price predictions. SVR and Stacking models delivered the best results, proving that ML can effectively automate used car pricing.



Future Scope

Deep learning, Flask/Streamlit deployment, real-time predictions



Improvements

Larger dataset, hyperparameter tuning, more features



Questions?

Happy to discuss the approach, models, or findings